

Trabajando con Hadoop y Hive



Ivana Sánchez Pérez

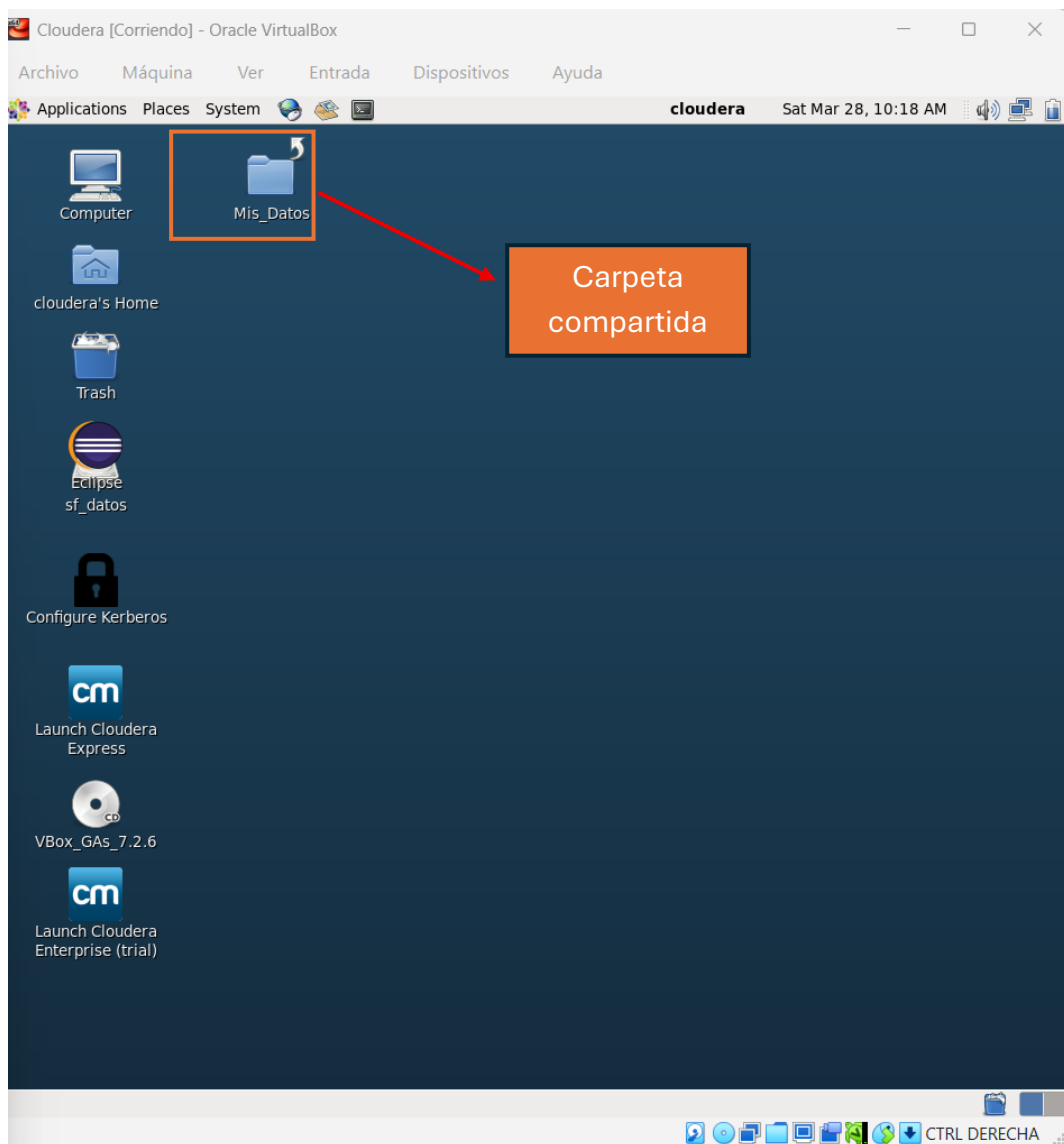
TRABAJANDO CON HADOOP Y HIVE

En este ejercicio se ha configurado un entorno de Big Data para organizar y consultar información bancaria (cuentas y clientes) utilizando Cloudera en una máquina virtual. A continuación, se van a explicar los pasos realizados de forma sencilla.

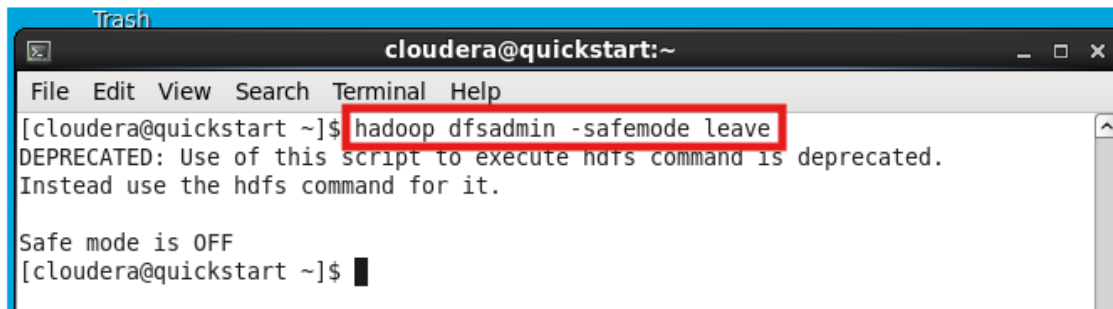
1. Preparación del Entorno

Lo primero que tenemos que hacer es poner en marcha nuestra "caja de herramientas".

- **Configuración Inicial:** Instalación de Cloudera en VirtualBox y configuración de una carpeta compartida llamada sf_datos para pasar los archivos de mi ordenador a la máquina virtual.



- **Desbloqueo del Sistema:** Hadoop (el motor que guarda los datos) viene "protegido" en su primer arranque por defecto. Para desactivar este 'modo seguro' y poder escribir datos en él, simplemente ejecutaremos el comando ***hadoop dfsadmin -safemode leave*** en el terminal.



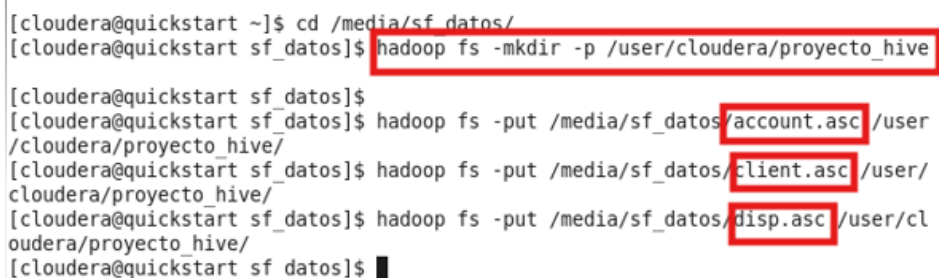
```
Trash
cloudera@quickstart:~
File Edit View Search Terminal Help
[cloudera@quickstart ~]$ hadoop dfsadmin -safemode leave
DEPRECATED: Use of this script to execute hdfs command is deprecated.
Instead use the hdfs command for it.

Safe mode is OFF
[cloudera@quickstart ~]$
```

2. Moviendo los datos al "Almacén" (HDFS)

Hive, que es la herramienta que usamos para hacer tablas, no puede leer archivos directamente de nuestra carpeta de Windows. Necesita que los datos estén dentro de su propio sistema de archivos HDFS.

- **Creación de un directorio**→ En el terminal, desde la carpeta compartida (`cd /media/sf_datos`) crearemos un directorio en el sistema de Hadoop al que llamaremos `proyecto_hive`.
- **Subida de archivos**→ Posteriormente moveremos los tres archivos necesarios: `account.asc` (cuentas), `client.asc` (clientes) y `disp.asc` (disposiciones) al nuevo directorio de Hadoop.



```
[cloudera@quickstart ~]$ cd /media/sf_datos/
[cloudera@quickstart sf_datos]$ hadoop fs -mkdir -p /user/cloudera/proyecto_hive

[cloudera@quickstart sf_datos]$
[cloudera@quickstart sf_datos]$ hadoop fs -put /media/sf_datos/account.asc /user/
/cloudera/proyecto_hive/
[cloudera@quickstart sf_datos]$ hadoop fs -put /media/sf_datos/client.asc /user/
/cloudera/proyecto_hive/
[cloudera@quickstart sf_datos]$ hadoop fs -put /media/sf_datos/disp.asc /user/cl
oudera/proyecto_hive/
[cloudera@quickstart sf_datos]$
```

3. Ejercicio 1: Creación de la Tabla Interna (Fragmentada)

El objetivo aquí es crear una tabla para las cuentas bancarias, pero organizada por el campo "frequency" (frecuencia).

- **Creación de la tabla final:** Entramos en Hive y creamos la tabla cuentas_internas. Como el archivo original usa puntos y comas (;) para separar los datos, a veces da errores, por lo que usamos el código especial \073 para que el sistema lo entienda perfectamente.

```
cloudera@quickstart:/media/sf_datos
File Edit View Search Terminal Help
[cloudera@quickstart sf_datos]$ hive

Logging initialized using configuration in file:/etc/hive/conf.dist/hive-log4j.properties
WARNING: Hive CLI is deprecated and migration to Beeline is recommended.
hive> CREATE TABLE cuentas_internas (
  > account_id INT,
  > district_id INT,
  > date STRING
  > )
  > PARTITIONED BY (frequency STRING)
  > ROW FORMAT DELIMITED
  > FIELDS TERMINATED BY '\073'
  > STORED AS TEXTFILE;
OK
Time taken: 2.348 seconds
hive> █
```

- **La Tabla Temporal:** Hive no sabe repartir los datos de un archivo plano directamente en carpetas organizadas. Por eso, crearemos primero una "tabla de paso" o temporal y cargaremos el archivo account.asc a esta tabla intermedia

```
cloudera@quickstart:/media/sf_datos
File Edit View Search Terminal Help
[cloudera@quickstart sf_datos]$ hive

Logging initialized using configuration in file:/etc/hive/conf.dist/hive-log4j.properties
WARNING: Hive CLI is deprecated and migration to Beeline is recommended.
hive> CREATE TABLE tmp_cuentas (
  > id INT,
  > dist INT,
  > freq STRING,
  > fec STRING
  > )
  > ROW FORMAT DELIMITED FIELDS TERMINATED BY '\073';
OK
Time taken: 0.806 seconds
hive> █

hive> LOAD DATA INPATH '/user/cloudera/proyecto_hive/account.asc' INTO TABLE tmp_cuentas;
Loading data to table default.tmp_cuentas
Table default.tmp_cuentas stats: [numFiles=1, totalSize=155356]
OK
Time taken: 1.245 seconds
hive> █
```

- **Fragmentación de datos:** Aquí es donde Hive leerá la tabla temporal y, según el valor de la columna freq, guardará cada fila en su carpeta correspondiente de la tabla fina. Esto se llama **particionamiento dinámico**.

Habilitamos el modo dinámico para que cree las carpetas

```
cloudera@quickstart:/media/sf_datos
File Edit View Search Terminal Help
hive> SET hive.exec.dynamic.partition=true;
hive> SET hive.exec.dynamic.partition.mode=nonstrict;
```

E insertamos los datos en la tabla particionada

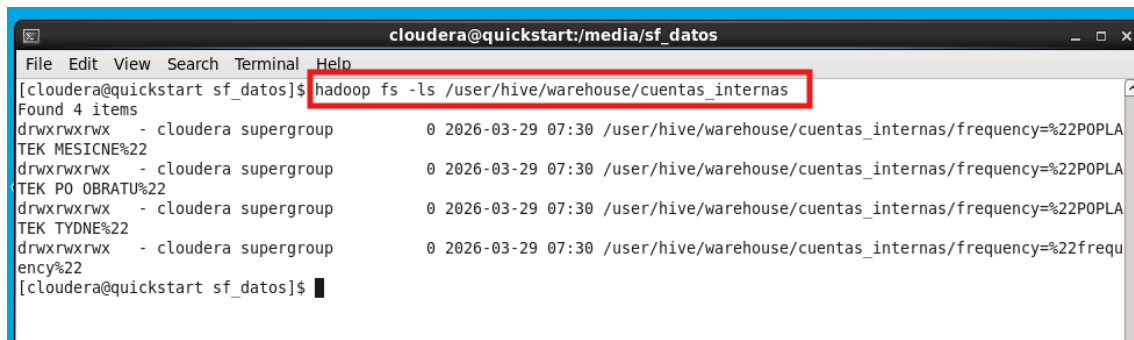
```
cloudera@quickstart:/media/sf_datos
File Edit View Search Terminal Help
hive> INSERT OVERWRITE TABLE cuentas_internas PARTITION(frequency)
> SELECT id, dist, fec, freq FROM tmp cuentas;
```

```
cloudera@quickstart:/media/sf_datos
File Edit View Search Terminal Help
Query ID = cloudera_20260329073030_43eb02be-4948-4ce9-8cb5-36d06bcf30d1
Total jobs = 3
Launching Job 1 out of 3
Number of reduce tasks is set to 0 since there's no reduce operator
Starting Job = job_1774792723791_0001, Tracking URL = http://quickstart.cloudera:8088/proxy/application_1774792723791_0001/
Kill Command = /usr/lib/hadoop/bin/hadoop job -kill job_1774792723791_0001
Hadoop job information for Stage-1: number of mappers: 1; number of reducers: 0
2026-03-29 07:30:47,689 Stage-1 map = 0%, reduce = 0%
2026-03-29 07:30:57,782 Stage-1 map = 100%, reduce = 0%, Cumulative CPU 2.7 sec
MapReduce Total cumulative CPU time: 2 seconds 700 msec
Ended Job = job_1774792723791_0001
Stage-4 is selected by condition resolver.
Stage-3 is filtered out by condition resolver.
Stage-5 is filtered out by condition resolver.
Moving data to: hdfs://quickstart.cloudera:8020/user/hive/warehouse/cuentas_internas/.hive-staging_hive_2026-03-29_07-30-29_079_8924616470769511118-1/-ext-10000
Loading data to table default.cuentas_internas partition (frequency=null)
Time taken for load dynamic partitions : 1950
Loading partition {frequency="POPLATEK TYDNE"}
Loading partition {frequency="POPLATEK MESICNE"}
Loading partition {frequency="frequency"}
Loading partition {frequency="POPLATEK PO OBRATU"}
Time taken for adding to write entity : 4
Partition default.cuentas_internas{frequency="POPLATEK MESICNE"} stats: [numFiles=1, numRows=4167, totalSize=60728, rawDataSize=56561]
Partition default.cuentas_internas{frequency="POPLATEK PO OBRATU"} stats: [numFiles=1, numRows=93, totalSize=1367, rawDataSize=1274]
Partition default.cuentas_internas{frequency="POPLATEK TYDNE"} stats: [numFiles=1, numRows=240, totalSize=3508, rawDataSize=3268]
Partition default.cuentas_internas{frequency="frequency"} stats: [numFiles=1, numRows=1, totalSize=13, rawDataSize=12]
MapReduce Jobs Launched:
Stage-Stage-1: Map: 1 Cumulative CPU: 2.7 sec HDFS Read: 159604 HDFS Write: 65960 SUCCESS
Total MapReduce CPU Time Spent: 2 seconds 700 msec
OK
Time taken: 33.757 seconds
```

- **Comprobación:** Al terminar, hacemos la comprobación de que hive ha creado las carpetas físicas en el disco duro virtual para cada tipo de frecuencia

```
hive> SHOW PARTITIONS cuentas_internas;
OK
frequency=%22POPLATEK MESICNE%22
frequency=%22POPLATEK PO OBRATU%22
frequency=%22POPLATEK TYDNE%22
frequency=%22frequency%22
Time taken: 0.189 seconds, Fetched: 4 row(s)
hive>
```

También comprobamos en el terminal para poder verlo físicamente.

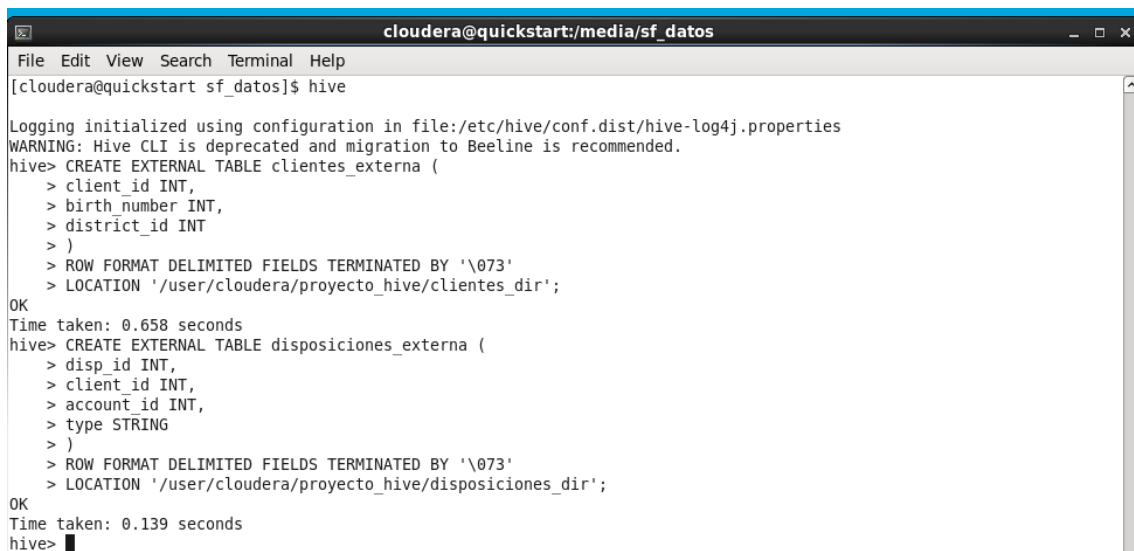


```
cloudera@quickstart:/media/sf_datos
File Edit View Search Terminal Help
[cloudera@quickstart sf_datos]$ hadoop fs -ls /user/hive/warehouse/cuentas_internas
Found 4 items
drwxrwxrwx - cloudera supergroup 0 2026-03-29 07:30 /user/hive/warehouse/cuentas_internas/frequency=%22POPLA
TEK MESICNE%22
drwxrwxrwx - cloudera supergroup 0 2026-03-29 07:30 /user/hive/warehouse/cuentas_internas/frequency=%22POPLA
TEK PO OBRATU%22
drwxrwxrwx - cloudera supergroup 0 2026-03-29 07:30 /user/hive/warehouse/cuentas_internas/frequency=%22POPLA
TEK TYDNE%22
drwxrwxrwx - cloudera supergroup 0 2026-03-29 07:30 /user/hive/warehouse/cuentas_internas/frequency=%22frequ
ency%22
[cloudera@quickstart sf_datos]$
```

4- Ejercicio 2: Crear Tablas Externas

Para los archivos de clientes y disposiciones, el ejercicio pedía crear tablas externas.

- **¿Qué significa "externa"?:** A diferencia de la tabla anterior, si borramos estas tablas en Hive, los archivos originales no se borran del almacén (HDFS). Son más seguras para datos compartidos.
- **Ejecución:** Creamos clientes_externa y disposiciones_externa apuntando a las carpetas donde guardamos los archivos .asc.

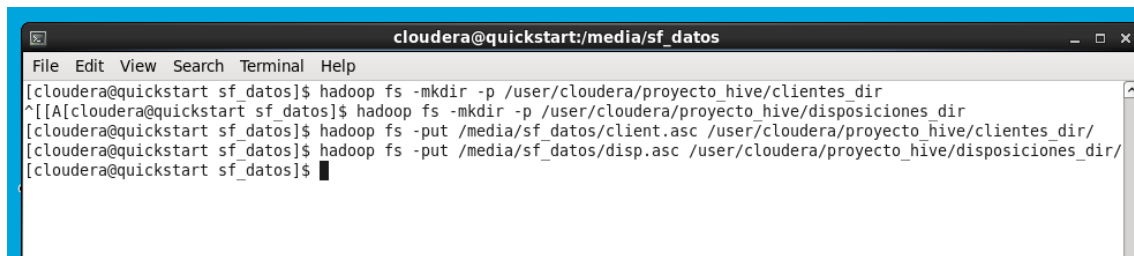


```
cloudera@quickstart:/media/sf_datos
File Edit View Search Terminal Help
[cloudera@quickstart sf_datos]$ hive
Logging initialized using configuration in file:/etc/hive/conf.dist/hive-log4j.properties
WARNING: Hive CLI is deprecated and migration to Beeline is recommended.
hive> CREATE EXTERNAL TABLE clientes_externa (
  > client_id INT,
  > birth_number INT,
  > district_id INT
  > )
  > ROW FORMAT DELIMITED FIELDS TERMINATED BY '\073'
  > LOCATION '/user/cloudera/proyecto_hive/clientes_dir';
OK
Time taken: 0.658 seconds
hive> CREATE EXTERNAL TABLE disposiciones_externa (
  > disp_id INT,
  > client_id INT,
  > account_id INT,
  > type STRING
  > )
  > ROW FORMAT DELIMITED FIELDS TERMINATED BY '\073'
  > LOCATION '/user/cloudera/proyecto_hive/disposiciones_dir';
OK
Time taken: 0.139 seconds
hive>
```

5- Ejercicio 3: La Consulta (JOIN)

Para terminar la tarea, deberemos juntar toda la información: saber qué cliente tiene qué cuenta y qué rol tiene ella (dueño o usuario), es decir, relacionar los clientes con sus cuentas.

- **Preparación:** crearemos carpetas a los archivos .asc para que las tablas externas puedan leerlos y el comando SELECT pueda mostrarlos.

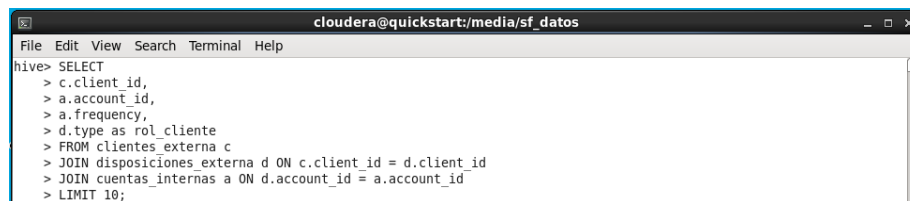


```

cloudera@quickstart:/media/sf_datos
File Edit View Search Terminal Help
[cloudera@quickstart sf_datos]$ hadoop fs -mkdir -p /user/cloudera/proyecto_hive/clientes_dir
^[[A[cloudera@quickstart sf_datos]$ hadoop fs -mkdir -p /user/cloudera/proyecto_hive/disposiciones_dir
[cloudera@quickstart sf_datos]$ hadoop fs -put /media/sf_datos/client.asc /user/cloudera/proyecto_hive/clientes_dir/
[cloudera@quickstart sf_datos]$ hadoop fs -put /media/sf_datos/disp.asc /user/cloudera/proyecto_hive/disposiciones_dir/
[cloudera@quickstart sf_datos]$

```

- **El resultado (JOIN):** Ejecutaremos una consulta (SELECT) que "unirá" las tres tablas. El sistema trabajó durante unos segundos y me mostró los primeros 10 resultados, confirmando que todo el sistema funciona correctamente.



```

cloudera@quickstart:/media/sf_datos
File Edit View Search Terminal Help
hive> SELECT
> c.client_id,
> a.account_id,
> a.frequency,
> d.type as rol_cliente
> FROM clientes externa c
> JOIN disposiciones externa d ON c.client_id = d.client_id
> JOIN cuentas_internas a ON d.account_id = a.account_id
> LIMIT 10;

```



```

cloudera@quickstart:/media/sf_datos
File Edit View Search Terminal Help
2026-03-29 07:59:30 Dump the side-table for tag: 1 with group count: 4500 into file: file:/tmp/cloudera/8788b78a-3894-4902-877b-2ca9ae387252/hive_2026-03-29_07-59-19_484_3275741027953634205-1/-local-10005/HashTable-Stage-5/MapJoin-mapfile01--.hashtable
2026-03-29 07:59:30 Uploaded 1 File to: file:/tmp/cloudera/8788b78a-3894-4902-877b-2ca9ae387252/hive_2026-03-29_07-59-19_484_3275741027953634205-1/-local-10005/HashTable-Stage-5/MapJoin-mapfile01--.hashtable (100482 bytes)
2026-03-29 07:59:30 Dump the side-table for tag: 0 with group count: 5369 into file: file:/tmp/cloudera/8788b78a-3894-4902-877b-2ca9ae387252/hive_2026-03-29_07-59-19_484_3275741027953634205-1/-local-10005/HashTable-Stage-5/MapJoin-mapfile10--.hashtable
2026-03-29 07:59:30 Uploaded 1 File to: file:/tmp/cloudera/8788b78a-3894-4902-877b-2ca9ae387252/hive_2026-03-29_07-59-19_484_3275741027953634205-1/-local-10005/HashTable-Stage-5/MapJoin-mapfile10--.hashtable (107804 bytes)
2026-03-29 07:59:30 End of local task; Time Taken: 1.747 sec.
Execution completed successfully
MapredLocal task succeeded
Launching Job 1 out of 1
Number of reduce tasks is set to 0 since there's no reduce operator
Starting Job = job_1774792723791_0003, Tracking URL = http://quickstart.cloudera:8088/proxy/application_1774792723791_0003/
Kill Command = /usr/lib/hadoop/bin/hadoop job -kill job_1774792723791_0003
Hadoop job information for Stage-5: number of mappers: 1; number of reducers: 0
2026-03-29 07:59:45,416 Stage-5 map = 0%, reduce = 0%
2026-03-29 07:59:54,144 Stage-5 map = 100%, reduce = 0%, Cumulative CPU 3.73 sec
MapReduce Total cumulative CPU time: 3 seconds 730 msec
Ended Job = job_1774792723791_0003
MapReduce Jobs Launched:
Stage-Stage-5: Map: 1 Cumulative CPU: 3.73 sec HDFS Read: 13456 HDFS Write: 319 SUCCESS
Total MapReduce CPU Time Spent: 3 seconds 730 msec
OK
1 1 "POPLATEK MESICNE" "OWNER"
2 2 "POPLATEK MESICNE" "OWNER"
3 2 "POPLATEK MESICNE" "DISPONENT"
4 3 "POPLATEK MESICNE" "OWNER"
5 3 "POPLATEK MESICNE" "DISPONENT"
6 4 "POPLATEK MESICNE" "OWNER"
7 5 "POPLATEK MESICNE" "OWNER"
8 6 "POPLATEK MESICNE" "OWNER"
9 7 "POPLATEK MESICNE" "OWNER"
10 8 "POPLATEK MESICNE" "OWNER"
Time taken: 38.86 seconds, Fetched: 10 row(s)
hive>

```

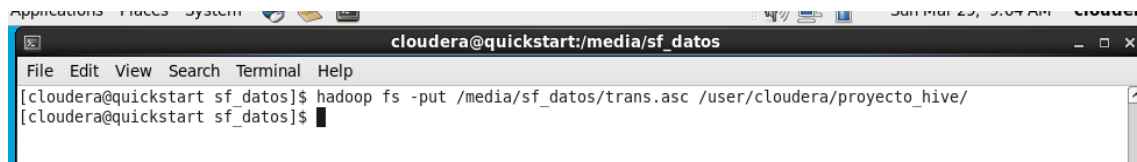
6- Ejercicio 4: ejemplo de particionado y bucketing

Para este ejercicio hemos elegido el archivo de Transacciones (trans.asc) porque es el más voluminoso y, hemos aplicado una técnica doble:

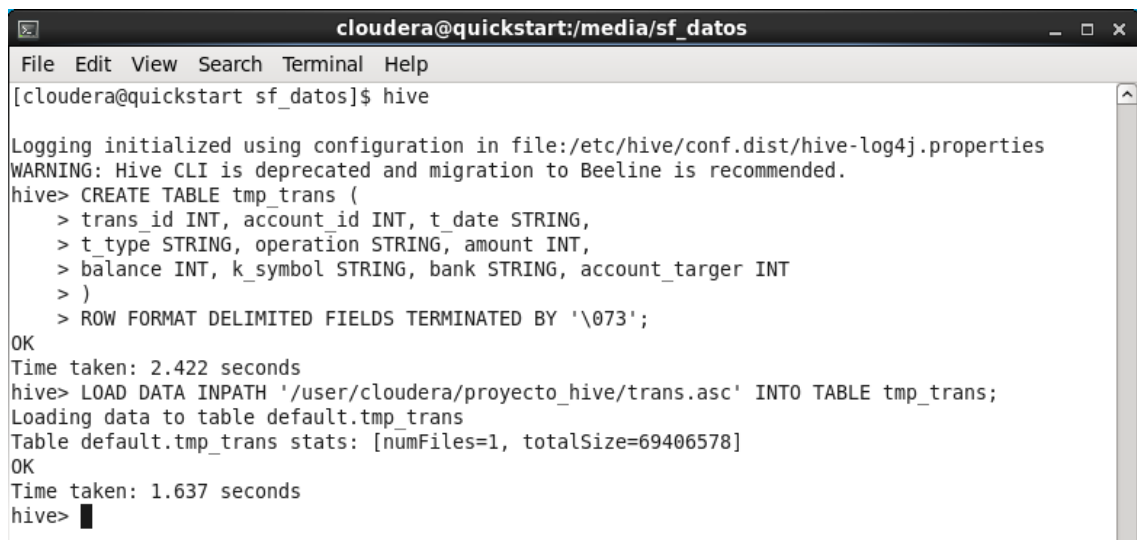
- 1- Particionado por Tipo (t_type): Dividimos las transacciones en carpetas según si son ingresos, retiradas, etc.
- 2- Bucketing por cuenta (account_id): dentro de cada carpeta, repartimos los datos en 4 “cubos” o ficheros basándonos en el número de cuenta.

PASOS REALIZADOS EN HIVE

- **PASO 1. Tabla de carga:** subimos el archivo trans.asc a HDFS y lo cargamos en una tabla temporal llamada tmp_trans para que Hive pueda leer los datos brutos



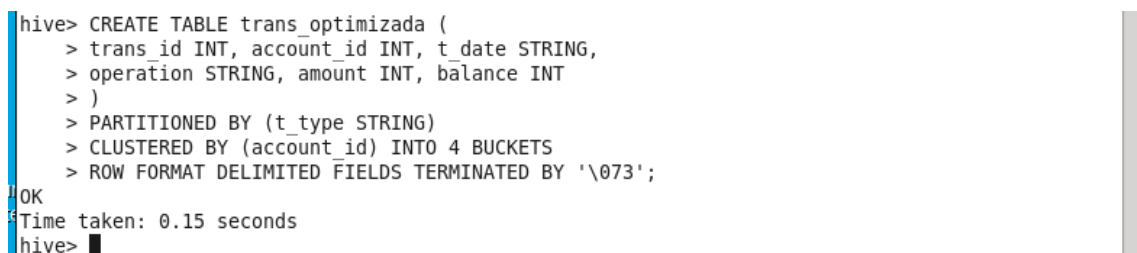
```
cloudera@quickstart:/media/sf_datos
File Edit View Search Terminal Help
[cloudera@quickstart sf_datos]$ hadoop fs -put /media/sf_datos/trans.asc /user/cloudera/proyecto_hive/
[cloudera@quickstart sf_datos]$
```



```
cloudera@quickstart:/media/sf_datos
File Edit View Search Terminal Help
[cloudera@quickstart sf_datos]$ hive

Logging initialized using configuration in file:/etc/hive/conf.dist/hive-log4j.properties
WARNING: Hive CLI is deprecated and migration to Beeline is recommended.
hive> CREATE TABLE tmp_trans (
  > trans_id INT, account_id INT, t_date STRING,
  > t_type STRING, operation STRING, amount INT,
  > balance INT, k_symbol STRING, bank STRING, account_targeter INT
  > )
  > ROW FORMAT DELIMITED FIELDS TERMINATED BY '\073';
OK
Time taken: 2.422 seconds
hive> LOAD DATA INPATH '/user/cloudera/proyecto_hive/trans.asc' INTO TABLE tmp_trans;
Loading data to table default.tmp_trans
Table default.tmp_trans stats: [numFiles=1, totalSize=69406578]
OK
Time taken: 1.637 seconds
hive>
```

- **Paso 2. Creación de la tabla optimizada.** Creamos una nueva tabla indicando que los datos deben ir separados por tipo y agrupados en 4 cubos por identificación de cuenta.



```
hive> CREATE TABLE trans_optimizada (
  > trans_id INT, account_id INT, t_date STRING,
  > operation STRING, amount INT, balance INT
  > )
  > PARTITIONED BY (t_type STRING)
  > CLUSTERED BY (account_id) INTO 4 BUCKETS
  > ROW FORMAT DELIMITED FIELDS TERMINATED BY '\073';
OK
Time taken: 0.15 seconds
hive>
```

- **Paso 3. El volcado de datos:** Al pasar los datos de la tabla temporal a la optimizada, Hive realizó un proceso de organización automática (MapReduce) para colocar cada transacción en su 'cajón' correspondiente.

```

cloudera@quickstart:/media/sf_datos
File Edit View Search Terminal Help
[cloudera@quickstart sf_datos]$ hive

Logging initialized using configuration in file:/etc/hive/conf.dist/hive-log4j.properties
WARNING: Hive CLI is deprecated and migration to Beeline is recommended.
hive> SET hive.exec.dynamic.partition=true;
hive> SET hive.exec.dynamic.partition.mode=nonstrict;
hive> SET hive.enforce.bucketing = true;
hive> INSERT OVERWRITE TABLE trans_optimizada PARTITION(t_type)
> SELECT trans_id, account_id, t_date, operation, amount, balance, t_type FROM tmp_trans;

```

```

cloudera@quickstart:/media/sf_datos
File Edit View Search Terminal Help
In order to set a constant number of reducers:
  set mapreduce.job.reduces=<number>
Starting Job = job_1774807531622_0001, Tracking URL = http://quickstart.cloudera:8088/proxy/app
location 1774807531622_0001/
Kill Command = /usr/lib/hadoop/bin/hadoop job -kill job_1774807531622_0001
Hadoop job information for Stage-1: number of mappers: 1; number of reducers: 4
2026-03-29 11:57:14,023 Stage-1 map = 0%, reduce = 0%
2026-03-29 11:57:22,230 Stage-1 map = 100%, reduce = 0%, Cumulative CPU 4.03 sec
2026-03-29 11:57:44,801 Stage-1 map = 100%, reduce = 17%, Cumulative CPU 6.79 sec
2026-03-29 11:57:48,681 Stage-1 map = 100%, reduce = 33%, Cumulative CPU 12.44 sec
2026-03-29 11:57:55,604 Stage-1 map = 100%, reduce = 67%, Cumulative CPU 27.19 sec
2026-03-29 11:57:56,940 Stage-1 map = 100%, reduce = 75%, Cumulative CPU 32.79 sec
2026-03-29 11:57:59,386 Stage-1 map = 100%, reduce = 92%, Cumulative CPU 35.33 sec
2026-03-29 11:58:00,619 Stage-1 map = 100%, reduce = 100%, Cumulative CPU 37.39 sec
MapReduce Total cumulative CPU time: 37 seconds 390 msec
Ended Job = job_1774807531622_0001
Loading data to table default.trans_optimizada partition (t_type=null)
  Time taken for load dynamic partitions : 639
  Loading partition {t_type="PRIJEM"}
  Loading partition {t_type="VYDAJ"}
  Loading partition {t_type="VYBER"}
  Loading partition {t_type="type"}
  Time taken for adding to write entity : 3
Partition default.trans_optimizada{t_type="PRIJEM"} stats: [numFiles=4, numRows=405083, totalSize=15009074, rawDataSize=14603991]
Partition default.trans_optimizada{t_type="VYBER"} stats: [numFiles=4, numRows=16666, totalSize=648017, rawDataSize=631351]
Partition default.trans_optimizada{t_type="VYDAJ"} stats: [numFiles=4, numRows=634571, totalSize=25716113, rawDataSize=25081542]
Partition default.trans_optimizada{t_type="type"} stats: [numFiles=4, numRows=1, totalSize=31, rawDataSize=30]
MapReduce Jobs Launched:
Stage-Stage-1: Map: 1 Reduce: 4 Cumulative CPU: 37.39 sec HDFS Read: 69429294 HDFS Write: 41374287 SUCCESS
Total MapReduce CPU Time Spent: 37 seconds 390 msec
OK
Time taken: 62.362 seconds

```

COMPARATIVA DE TIEMPOS

Para demostrar la utilidad de todo esto, hemos realizado la misma consulta (sumar gastos de una cuenta específica) en ambas tablas:

CONSULTA A → SOBRE LA TABLA OPTIMIZADA

```
cloudera@quickstart:/media/sf_datos
File Edit View Search Terminal Help
hive> SELECT account_id, SUM(amount) FROM trans_optimizada
      WHERE t_type = "VYDAJ" GROUP BY account_id LIMIT 10;
Query ID = cloudera_20260329120101_436361fb-6f23-4ddd-8aae-a0613eea6d5c
Total jobs = 1
Launching Job 1 out of 1
Number of reduce tasks not specified. Estimated from input data size: 1
In order to change the average load for a reducer (in bytes):
  set hive.exec.reducers.bytes.per.reducer=<number>
In order to limit the maximum number of reducers:
  set hive.exec.reducers.max=<number>
In order to set a constant number of reducers:
  set mapreduce.job.reduces=<number>
Starting Job = job_1774807531622_0002, Tracking URL = http://quickstart.cloudera:8088/proxy/application_1774807531622_0002/
Kill Command = /usr/lib/hadoop/bin/hadoop job -kill job_1774807531622_0002
Hadoop job information for Stage-1: number of mappers: 1; number of reducers: 1
2026-03-29 12:01:56,490 Stage-1 map = 0%, reduce = 0%
2026-03-29 12:02:02,310 Stage-1 map = 100%, reduce = 0%, Cumulative CPU 1.68 sec
2026-03-29 12:02:09,272 Stage-1 map = 100%, reduce = 100%, Cumulative CPU 2.73 sec
MapReduce Total cumulative CPU time: 2 seconds 730 msec
Ended Job = job_1774807531622_0002
MapReduce Jobs Launched:
Stage-Stage-1: Map: 1 Reduce: 1 Cumulative CPU: 2.73 sec HDFS Read: 25725505 HDFS Write: 91 SUCCESS
Total MapReduce CPU Time Spent: 2 seconds 730 msec
OK
1      180830
2      1336928
3      121955
4      158608
5      69388
6      299922
7      561760
8      820124
9      336238
10     490641
Time taken: 20.629 seconds, Fetched: 10 row(s)
hive>
```

CONSULTA B → SOBRE LA TABLA TEMPORAL SIN OPTIMIZAR

```
cloudera@quickstart:/media/sf_datos
File Edit View Search Terminal Help
hive> SELECT account_id, SUM(amount) FROM tmp_trans
      WHERE t_type = "VYDAJ" GROUP BY account_id LIMIT 10;
Query ID = cloudera_20260329120404_a8e281c7-51c6-4400-a29d-d0a970d2fe2d
Total jobs = 1
Launching Job 1 out of 1
Number of reduce tasks not specified. Estimated from input data size: 1
In order to change the average load for a reducer (in bytes):
  set hive.exec.reducers.bytes.per.reducer=<number>
In order to limit the maximum number of reducers:
  set hive.exec.reducers.max=<number>
In order to set a constant number of reducers:
  set mapreduce.job.reduces=<number>
Starting Job = job_1774807531622_0003, Tracking URL = http://quickstart.cloudera:8088/proxy/application_1774807531622_0003/
Kill Command = /usr/lib/hadoop/bin/hadoop job -kill job_1774807531622_0003
Hadoop job information for Stage-1: number of mappers: 1; number of reducers: 1
2026-03-29 12:04:42,471 Stage-1 map = 0%, reduce = 0%
2026-03-29 12:04:48,814 Stage-1 map = 100%, reduce = 0%, Cumulative CPU 2.89 sec
2026-03-29 12:04:56,293 Stage-1 map = 100%, reduce = 100%, Cumulative CPU 4.13 sec
MapReduce Total cumulative CPU time: 4 seconds 130 msec
Ended Job = job_1774807531622_0003
MapReduce Jobs Launched:
Stage-Stage-1: Map: 1 Reduce: 1 Cumulative CPU: 4.13 sec HDFS Read: 69416490 HDFS Write: 91 SUCCESS
Total MapReduce CPU Time Spent: 4 seconds 130 msec
OK
1      180830
2      1336928
3      121955
4      158608
5      69388
6      299922
7      561760
8      820124
9      336238
10     490641
Time taken: 22.597 seconds, Fetched: 10 row(s)
hive>
```

Podemos observar que en la tabla con particionado y Bucket la consulta fue mucho más rápido porque Hive sabía exactamente en qué carpeta de 'tipo' y en qué 'cubo' de cuenta buscar, ignorando el resto de la información innecesaria.

Sin embargo, en la tabla normal (sin optimizar), la consulta tardó más, pues el sistema tuvo que leer todo el archivo de transacciones de arriba abajo para encontrar los datos.

CONCLUSIÓN

Tras completar los cuatro ejercicios, se han alcanzado los siguientes hitos técnicos:

- **Optimización del almacenamiento:** Mediante el particionamiento dinámico en la tabla cuentas_internas, hemos logrado que Hadoop organice la información físicamente en carpetas según la frecuencia. Esto permite que las consultas futuras sean mucho más rápidas al no tener que escanear todo el conjunto de datos.
- **Flexibilidad de datos:** El uso de tablas externas demuestra cómo Hive puede trabajar con archivos compartidos sin necesidad de "adueñarse" de ellos, manteniendo la integridad de los archivos originales en HDFS.
- **Integración de información:** La consulta JOIN final confirma que la estructura diseñada es funcional, permitiendo cruzar datos de diferentes fuentes (tablas internas y externas) para obtener información valiosa de negocio, como la relación entre clientes, cuentas y sus roles.

El sistema queda totalmente operativo y validado.